

1. OBJECTIF PRINCIPAL

Développer **Fondora**, une application web de gestion financière personnelle complète (style Finary), déployée sur Vercel, avec code source sur GitHub privé et données sécurisées sur Supabase. L'application permet de gérer budgets, dépenses, investissements (actions, crypto, immobilier), prévoir les finances mensuelles, et visualiser tout ça avec des graphiques modernes.

2. INFORMATIONS, DONNÉES ET CONTRAINTES FOURNIES

Plateforme & Technique

- Application **web** accessible depuis un lien de partage (Vercel)
- Code source sur **GitHub privé**
- Plusieurs utilisateurs possibles, chaque utilisateur a ses données **100% privées et sécurisées**
- Compte et connexion obligatoires (email + mot de passe + Google Sign-In)
- **Saisie manuelle** des transactions (pas de connexion bancaire pour des raisons de sécurité)
- Import CSV possible en plus de la saisie manuelle

Stack Technique Validée

- **Frontend** : React + Vite + Tailwind CSS v4 + Recharts
- **Backend/Auth/DB** : Supabase (Row Level Security activé sur toutes les tables)
- **APIs** :
 - Finnhub (actions/ETF, cours temps réel + infos entreprise) via Edge Function sécurisée
 - CoinGecko (crypto, appel direct depuis navigateur, pas de clé nécessaire)
 - Frankfurter (taux de change EUR/USD, gratuit sans clé)
- **Export** : SheetJS prévu (étape 14)
- **Hébergement** : Vercel (déploiement automatique à chaque git push)

Charte Graphique

- Couleurs : Bleu marine (#0a1f33), navy-light (#122a44), vert émeraude (#10b981), emerald-light (#34d399), gris clair (#f1f5f9)
- Style épuré, données en avant, thème clair avec accents navy/emerald
- Tailwind CSS avec @theme custom dans src/index.css

Fonctionnalités Demandées

- Gestion de comptes (courant, épargne, crédit, PEA, assurance vie, crypto, immobilier)

- Transactions manuelles + import CSV
- Catégories entièrement personnalisables (avec catégories par défaut)
- Dépenses récurrentes automatiques (case à cocher)
- Budget par catégorie avec alertes de dépassement
- Graphiques : Sankey, donut, barres comparatives, courbes d'évolution, area chart
- Projections de fin de mois + taux d'épargne
- Objectifs d'épargne mensuels
- Investissements : PEA/CTO (Finnhub), Crypto (CoinGecko), Immobilier (calculs rentabilité)
- Assurance vie (saisie manuelle)
- Multi-devises EUR/USD avec taux de change automatique
- Export données + graphiques (Excel)
- Comparaison patrimoine vs données INSEE publiques
- Top 10 cryptos mondiales (CoinGecko)
- Simulateur d'achat crypto
- Historique de valorisation du portefeuille avec graphique (7j/30j/1an/tout)

Informations Compte Utilisateur

- GitHub : mathcast-30
- Repo : <https://github.com/mathcast-30/Fondora.git>
- Supabase Project URL : <https://ptcdwzozambankspjwes.supabase.co>
- Supabase Publishable Key : sb_publishable_TD7gNNWI3bPIPFtsVh06NA_lbE6jThr
- User ID principal (pour tests SQL) : 3deee05f-dc70-4dda-974e-115b0c5f28bd
- Email utilisateur test : matheo.helou@gmail.com

3. DÉCISIONS ET STRUCTURES VALIDÉES

Architecture des dossiers

src/

components/

Modal.jsx

Sidebar.jsx

Layout.jsx

CompteCard.jsx

BudgetBar.jsx

SankeyChart.jsx

DonutChart.jsx ← modifié pour être réutilisable (props: valueKey, nameKey, colorKey, libelleCentre)

BarChartComparatif.jsx

LineChartSolde.jsx

StatCard.jsx

EvolutionPatrimoineChart.jsx

LegendeAllocation.jsx

DividendesCard.jsx

SimulateurAchatCrypto.jsx

TopCryptoTable.jsx

BienImmobilierCard.jsx

FormBienImmobilier.jsx ← formulaire en 4 étapes (Général/Crédit/Charges/Locatif)

ComparaisonINSEE.jsx

ObjectifEpargneCard.jsx

ProjectionCard.jsx

pages/

Login.jsx

Signup.jsx

Synthese.jsx ← page d'accueil (projection, objectif épargne, liste comptes)

Budget.jsx ← transactions, budgets, Sankey, donut

Patrimoine.jsx ← comptes + total consolidé (actions+crypto+immo) + INSEE

Investir.jsx ← 3 onglets : Actions/ETF | Crypto | Immobilier

Analyse.jsx ← stats comparatives, barres, courbes

Parametres.jsx ← vide pour l'instant

hooks/

useComptes.js

useCategories.js

useTransactions.js

useBudgets.js

useObjectifEpargne.js

usePositions.js

useCoursBourse.js

useHistoriquePatrimoine.js

usePositionsCrypto.js

useCoursCrypto.js

useTopCrypto.js

useBiensImmobiliers.js

useDividendes.js

useTransactionsPeriode.js

context/

AuthContext.jsx ← user, signUp, signIn, signInWithGoogle, signOut

lib/

supabase.js ← createClient avec variables VITE_

dateUtils.js ← regrouperParMois()

projection.js ← calculerProjection()

calculImmo.js ← calculerMensualiteCredit(), calculerRentabilite()

cryptosCourantes.js ← CRYPTOS_COURANTES (liste de 15 cryptos)

donneesInsee.js ← PATRIMOINE_MEDIAN_FRANCE, getTrancheParAge()

App.jsx ← BrowserRouter + toutes les routes + ProtectedRoute

main.jsx ← AuthProvider wrapping App

index.css ← @import "tailwindcss" + @theme avec couleurs custom

Tables Supabase créées (toutes avec RLS activé)

sql

-- 1. comptes

id, user_id, nom, type, solde, devise, couleur, created_at

-- 2. categories

id, user_id, nom, type ('depense'|'revenu'), couleur, created_at

-- 3. transactions

id, user_id, compte_id, categorie_id, description, montant, type, date,
recurrente (bool), jour_recurrence (int), recurrence_groupe_id (uuid), created_at

-- 4. budgets

id, user_id, categorie_id, montant_max, mois, annee, created_at
UNIQUE(categorie_id, mois, annee)

-- 5. objectifs_epargne

id, user_id, montant_cible, mois, annee, created_at
UNIQUE(user_id, mois, annee)

-- 6. historique_patrimoine

id, user_id, date, valeur_positions, created_at
UNIQUE(user_id, date)

-- 7. positions_financieres

id, user_id, symbole, nom, quantite, prix_achat_moyen, devise, type_compte, created_at

-- 8. dividendes

id, user_id, position_id, montant, date, created_at

-- 9. positions_crypto

id, user_id, coin_id, symbole, nom, quantite, prix_achat_moyen (en EUR), created_at

-- 10. biens_immobiliers

id, user_id, nom, adresse, type_bien, statut,
prix_achat, valeur_actuelle, date_achat,
montant_credit, taux_credit, duree_credit_mois, mensualite_credit,
assurance_emprunteur_annuelle, taxe_foncieres_annuelle, charges_copropriete_annuelle,
assurance_habitation_annuelle, frais_gestion_annuelle, travaux_annuels,
loyer_mensuel, taux_vacance, created_at

Edge Function Supabase déployée

- Nom : cours-bourse
- Fichier : supabase/functions/cours-bourse/index.ts
- Secret configuré : FINNHUB_API_KEY
- Rôle : reçoit un symbole, appelle Finnhub (cours + profil entreprise), retourne { symbole, coursActuel, variationJour, nom, logo, secteur }

Trigger Supabase (catégories par défaut)

sql

```
-- Fonction creer_categories_par_defaut() + trigger on_auth_user_created
-- Crée 11 catégories automatiquement à chaque nouvelle inscription
-- (8 dépenses + 3 revenus)
```

Fichier .env (local uniquement, jamais sur GitHub)

VITE_SUPABASE_URL=https://ptcdwzozambankspjwes.supabase.co

VITE_SUPABASE_ANON_KEY=sb_publishable_TD7gNNWI3bPIPFtsVh06NA_lbE6jThr

Navigation (Sidebar)

- Synthèse (/)
- Budget (/budget)
- Patrimoine (/patrimoine)
- Investir (/investir) → onglets internes : Actions & ETF | Crypto | Immobilier
- Analyse (/analyse)
- Paramètres (/parametres)

4. ÉTAPES COMPLÉTÉES

Étape	Statut	Contenu
1	✓	Fondations : GitHub, Vite/React, Tailwind, Supabase, Vercel
2	✓	Auth : inscription, connexion email, Google Sign-In, ProtectedRoute, RLS
3	✓	Charte graphique : Sidebar, Layout, pages placeholder
4	✓	Comptes : table, hook, CRUD, calcul patrimoine total
5	✓	Transactions : table, hook, catégories, récurrence automatique
6	✓	Budgets : table, hook, BudgetBar, alertes, Sankey, Donut

Étape	Statut	Contenu
7	✓	Analyse : hook multi-mois, barres comparatives, courbe solde, stats
8	✓	Projections, objectifs épargne, page Synthèse complète
9a	✓	Historique patrimoine, AreaChart avec sélecteur période
9b	✓	Donut allocation portefeuille + légende
9c	✓	Dividendes (saisie manuelle) + calcul rendement
9d	✓	Logos entreprises Finnhub + mise en page 2 colonnes
9e	✓	Module Crypto : simulateur achat, liste positions, cours CoinGecko
9f	✓	Comparaison INSEE (curseur âge + médiane France)
9g	✓	Top 10 cryptos mondiales (tableau capitalisation)
10	EN COURS	Immobilier : table créée, hook créé, BienImmobilierCard, FormBienImmobilier (4 étapes), calculImmo.js — fichier Investir.jsx corrigé (bug require → import fixé) — À TESTER

5. PROCHAINES ÉTAPES IMMÉDIATES

Immédiat : Valider l'étape 10

Vérifier que l'onglet Immobilier dans Investir fonctionne après la correction du bug `require()` → `import calculerRentabilite from '../lib/calculImmo'` fait en haut du fichier.

Tests à faire :

1. Onglet Immobilier s'affiche sans écran blanc
2. Bouton "Ajouter un bien" → formulaire 4 étapes fonctionne
3. Carte bien affiche correctement les indicateurs (cash-flow, rentabilité)
4. Page Patrimoine affiche le total consolidé (comptes + actions + crypto + immo)

Étape 11 : Assurance vie

- Table `assurances_vie` (saisie manuelle des montants + historique)
- Si contient des unités de compte (actions), afficher comme les positions financières
- Intégration dans Patrimoine (total consolidé)

Étape 12 : Module multi-devises

- Hook `useTauxChange` (API Frankfurter, gratuit)
- Conversion EUR/USD automatique partout où c'est pertinent

- Affichage du taux actuel dans Paramètres

Étape 13 : Page Paramètres

- Activer/désactiver les modules (Crypto, Immobilier, Assurance vie, PEA/CTO)
- Gestion des catégories (ajouter/supprimer/modifier les catégories personnalisées)
- Informations du profil utilisateur
- Préférences (devise principale, langue)

Étape 14 : Export Excel (SheetJS)

- Exporter les transactions en Excel
- Exporter les graphiques (Recharts → canvas → image dans Excel)
- Exporter le rapport mensuel complet

Étape 15 : Polish final

- Responsive mobile (sidebar collapsible sur petit écran)
- Animations/transitions douces sur les graphiques
- Messages d'erreur / états de chargement plus soignés
- Tests finaux multi-utilisateurs
- Variables d'environnement côté Vercel (ajouter VITE_SUPABASE_URL et VITE_SUPABASE_ANON_KEY dans les settings Vercel pour que le build en production fonctionne)

6. POINTS D'ATTENTION IMPORTANTS POUR LA SUITE

1. **Variables Vercel** : ajouter VITE_SUPABASE_URL et VITE_SUPABASE_ANON_KEY dans Vercel → Settings → Environment Variables pour que le déploiement en production fonctionne (pas juste en local)
2. **Import CSV** : prévu mais pas encore codé — nécessite une librairie de parsing CSV (PapaParse déjà dans les dépendances React disponibles) + une modal d'import dans Budget
3. **Clé Finnhub** : plan gratuit = 60 appels/minute. Si beaucoup de symboles dans le portefeuille, penser à ajouter un cache côté Edge Function
4. **Bug corrigé** : dans Investir.jsx, le require('../lib/calculImmo') dans le JSX a été remplacé par un import { calculerRentabilite } from '../lib/calculImmo' en haut du fichier + calcul de cashFlowTotal dans le corps du composant avant le return
5. **Catégories** : le trigger SQL ne s'applique qu'aux nouveaux utilisateurs. L'utilisateur existant (matheo.helou@gmail.com) a eu ses catégories insérées manuellement via SQL avec son UUID

6. **Google Sign-In** : configuré sur Google Cloud Console (projet "Fondora", client OAuth "Fondora Web") avec les redirect URIs Supabase. Fonctionne en local sur `http://localhost:5173`
 7. **Workflow Git** : `git add .` → `git commit -m "message"` → `git push` → Vercel redéploie automatiquement
-

7. COMMANDES UTILES À RETENIR

`bash`

Lancer en développement

`npm run dev`

Déployer une Edge Function Supabase

`supabase functions deploy cours-bourse`

Ajouter un secret Supabase

`supabase secrets set FINNHUB_API_KEY=ta_cle`

Pousser sur GitHub (déclenche redéploiement Vercel automatique)

`git add .`

`git commit -m "Description des changements"`

`git push`